

Operating System Simulator

Daniel Tebor, Matthew Watson, William Walker, Cody Hahn, Preston Dietz, Pranav Kartha, Jacob Suda

Abstract—Our paper details the design and implementation of an operating system simulator meant to demonstrate our understanding of the basic concepts of operating system design. The operating system package was written in Rust and contains several modules inside the simulated kernel, each representing a different controller or resource of the operating system. The package also includes IO modules for reading and writing the program instructions, data, and results. Operating system performance, including process turnaround time and burst time on the CPU was evaluated. This was done using different scheduling algorithms including FIFO and priority. Results showed that priority scheduling resulted in a faster turnaround time for processes with a higher priority.



1 INTRODUCTION

This report was written to explain an understanding and show the inner workings and decisions behind our operating system simulator. This was done through analysis of the simulator through various aspects and viewpoints of the simulator itself. These aspects and viewpoints include the design approach, implementations of the modules of the operating system, the simulation of the operating system itself, and data collected throughout the operation of the simulation regarding process timings and memory sizes. Each aspect listed is distinct, and combined, providing a complete understanding regarding the structure and function of the simulated operating system.

The team's approach regarding the simulation design stemmed from specifications and a flow chart provided by the professor. While the exact function of each section may not be fully synonymous to the provided materials, the complete system operates very closely to how a modern operating system would. The data provided through the program file flows through the system as it might in actuality. To enable this, each module of the system includes functions as if it were the actual component represented in the simulation.

Said modules, while they must work together, also have individual significant sections of code. These segments are imperative to the proper function of each module. Some of these modules are straightforward, while others are two to three times more complex. It can be assumed that the CPU module needed more attention to detail than the driver module, for example. The team had to be meticulous when developing these modules to assure accurate results and representation of real-world scenarios.

To test the accuracy of the simulation, a program file was provided. This program file contains 30 unique programs, with machine code level instructions. These programs have a set result and output, which is the basis for examining the correctness of the system. Before going through and testing the results of all 30 programs, the team used four unit-tests. These unit-tests were individual jobs from the program file, coded specifically for testing the CPU.

Once the unit-tests were completed to the team's satisfaction, the 30 programs were processed. They were processed through two different scheduling algorithms. Through the

team's design, as each program was loaded and processed, the timing and memory data was saved and tabulated in the system's final output. This data was used to analyze the system's effectiveness and would have been compared to a multi-core version of the project if there was more time.

Our report provides a clear explanation and insight into how the complete system operates. For example, the design approach contains an overview of the operating system itself through a flow diagram. Each component of the code package was dissected for examples of significant code segments. Examples of input and output were included in the analysis of the simulation. A compilation of the data found regarding turnaround time, burst time, and memory used by each program as it was processed is included, and the deductions on the data mentioned are elaborated on at the end of the report.

2 DESIGN APPROACH

Our system design follows the design shown in Figure 1. The driver, in charge of running the simulation, first invokes the loader, which loads the program file into our virtual disk. Then, the long-term scheduler queues as many programs as it can from the disk into memory. Next, the driver invokes the short-term scheduler, which uses a ready queue to dispatch the processes to the CPU. Now, the CPU, which has just received a process control block from the dispatcher, runs the process until it halts. It does this by cycling its data path cycle, consisting of instruction fetch, decode, and execute. When instruction execution is an IO instruction, the DMA channel is invoked by the CPU. After the program halts, the short-term scheduler dispatches the next program to be run onto the CPU. After all loaded programs have finished executing, control passes back to the driver which invokes the long-term scheduler to load the next batch of programs. This cycle continues until all jobs have been executed.

3 IMPLEMENTATION MODULES

The operating system is composed of multiple modules, each handling different requirements. These include the driver, loader, disk, memory, short-term scheduler, long-term scheduler, and central processing unit.

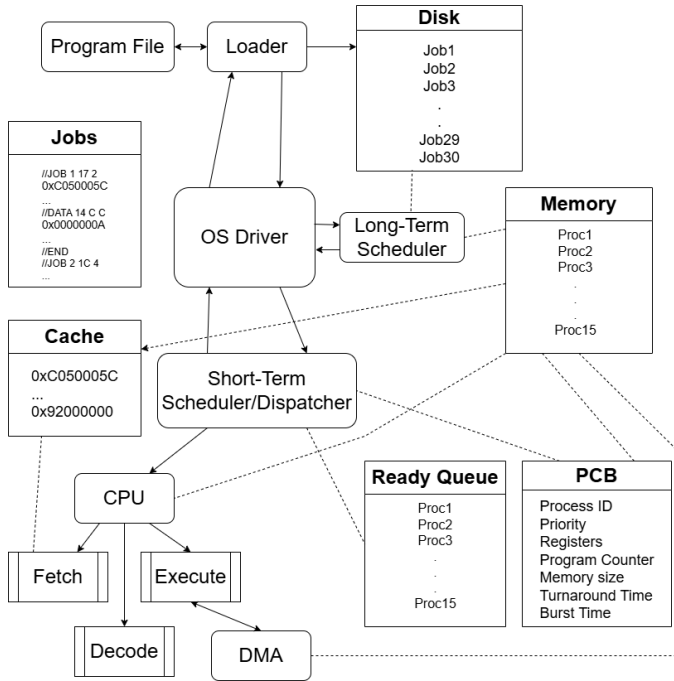


Fig. 1. The design of the Operating System

3.1 Driver

System initialization and orchestration is handled by the driver.

```
//const SCHEDULING_ALG: StsSchedulingAlg = StsSchedulingAlg::Fifo;
const SCHEDULING_ALG: StsSchedulingAlg = StsSchedulingAlg::Priority;

pub struct Driver {
    _cpu: Arc<Mutex<Cpu>>,
    disk: Rc<RefCell<Disk>>,
    memory: Arc<RwLock<Memory>>,
    lts: LongTermScheduler,
    sts: ShortTermScheduler,
}
```

The driver instantiates and stores the system resources including the central processing unit (CPU), disk, memory, long-term scheduler, and short-term scheduler. The CPU and memory are wrapped in an atomic reference counter and a mutex to orchestrate sharing between resources and threads.

Additionally, the scheduling algorithm used by the short-term scheduler is determined in the driver module. Finally, the driver is also responsible for the operating system simulation as detailed in Section 4.

3.2 Loader and Disk

The loader has two responsibilities: loading programs into the virtual disk and writing the virtual disk into a file.

3.2.1 Program and Disk Structure

The programs to be run on the operating system are written in 32-bit words represented as hexadecimal. As seen in Figure 1 Each program (job) is represented with a "/// JOB", "/// Data", and "/// END" header with data in-between.

The hexadecimal values after the job header represent the program ID, number of instructions, and priority. The

hexadecimal values after the data header represent the input buffer size, output buffer size, and temp buffer size.

A virtual disk is needed to store the programs read in from a program file. To store the programs, the virtual disk keeps an array of 32-bit words represented as u32 in Rust. The disk also stores a hashed map of program information to track the program IDs, priorities, and locations in the disk array.

```
const DISK_SIZE: usize = 2048;

#[derive(Clone)]
pub(crate) struct ProgramInfo {
    pub id: u32,
    pub priority: u32,
    pub instruction_buffer_size: usize,
    pub in_buffer_size: usize,
    pub out_buffer_size: usize,
    pub temp_buffer_size: usize,
    pub data_start_idx: usize,
}

pub(crate) struct Disk {
    program_map: HashMap<u32, ProgramInfo>,
    data: [u32; DISK_SIZE],
    current_data_idx: usize,
}
```

3.2.2 Loading Programs

Programs are loaded in from a program file line-by-line. Each line is processed as follows:

```
if line.starts_with("// JOB") {
    let job_info = &line[7..];
    let job_info: Vec<&str> = job_info.split_whitespace().collect();

    id = u32::from_str_radix(job_info[0], 16).unwrap();
    instruction_buffer_size = usize::from_str_radix(
        job_info[1], 16).unwrap();
    priority = u32::from_str_radix(job_info[2], 16).unwrap();
} else if line.starts_with("// Data") {
    let data_info = &line[8..];
    let data_info: Vec<&str> = data_info.split_whitespace().collect();

    in_buffer_size = usize::from_str_radix(data_info[0], 16).unwrap();
    out_buffer_size = usize::from_str_radix(data_info[1], 16).unwrap();
    temp_buffer_size = usize::from_str_radix(data_info[2], 16).unwrap();
} else if line.starts_with("// END") {
    disk.write_program(
        id,
        priority,
        instruction_buffer_size,
        in_buffer_size,
        out_buffer_size,
        temp_buffer_size,
        data.as_slice()
    );

    program_ids.push(id);
    data.clear();
} else {
    let line = line.trim();
    let value = u32::from_str_radix(&line[2..], 16).unwrap_or_else(|_| {
        panic!("Failed to parse hex value: {}", line)
    });

    data.push(value);
}
```

If a line is a job or data header, the parameters are extracted. Otherwise, the line is pushed to the data vector. If the header is an end header, the program is written to disk and the next program begins. The amount of space

Algorithm 1 Compute Disk Space Usage

Input: Instruction buffer size IB , Input buffer size InB , Output buffer size $OutB$, Temp buffer size TB
Output: Disk space usage D
 $D \leftarrow IB + InB + OutB + TB$
return D

each program uses in the disk can be computed as seen in Algorithm 1.

3.2.3 Writing Disk to File

Writing the virtual disk to a file is the inverse of loading programs into the disk. For each program in the disk, a job and data header are generated, and the program's data is appended line-by-line. Finally, the lines are written to a file.

3.3 Memory

To give the programs in the disk 'life' as processes, virtual memory is required.

```
const MEMORY_SIZE: usize = 1024;

pub(crate) struct ProcessControlBlock {
    pub program_counter: usize,
    pub registers: [u32; 16],
    pub state: ProcessState,

    id: u32,
    priority: u32,

    ...
}

pub(crate) struct Memory {
    pcb_map: HashMap<u32, Arc<Mutex<ProcessControlBlock>>>,
    data: [u32; MEMORY_SIZE],
    current_data_idx: usize,
}
```

Virtual memory is structured similarly to the virtual disk in Section 3.2. The memory has a data array of 1024 32-bit words and a hashed map of process control blocks (PCBs).

The PCB is a struct that stores runtime information for the process. When a program is loaded into memory as a process, a PCB is created to keep track of vital information including the process ID, priority, program counter, registers, state, address space in memory, and timing data.

Each process in memory is stored as a single block of data. The size of the block can be computed using Algorithm 1 since the process is created from a program.

Algorithm 2 Compute Physical Address

Input: Logical address L , Process start physical address S
Output: Physical address P
 $P \leftarrow L + S$
return P

Memory provides simple methods to read and write data given a physical address. There are also methods to read and write blocks of data given a starting and ending physical address. The PCB's memory start address field can be used to compute a physical address from a logical address as seen in Algorithm 2.

3.4 Long-Term Scheduler

The long-term scheduler (LTS) spawns processes in memory using programs from the disk.

```
pub(crate) struct LongTermScheduler {
    disk: Rc<RefCell<Disk>>,
    memory: Arc<RwLock<Memory>>,
    program_queue: VecDeque<u32>,
    unload_list: Vec<u32>,
}
```

A reference to the disk and memory are stored in the LTS for use. A program queue and an unload list are also maintained. The program queue is a queue of program IDs from the disk that will be loaded into memory as processes. The unload list keeps track of the process IDs for the processes in memory that need to be unloaded.

To schedule a program in the LTS a method called "enqueue_programs" is called, which takes a vector of program IDs and enqueues them to the program queue. Processes are spawned for the enqueued programs using the "step" and "batch_step" methods, which are as follows:

```
pub fn step(&mut self) -> Result<u32, &'static str> {
    let program_id = *self.program_queue.front().ok_or("No programs in queue")?;
    let disk = self.disk.borrow();

    let program_info = disk.get_info_for(program_id);
    let program_data = disk.read_data_for(&program_info);

    let mut memory = self.memory.write().unwrap();

    if memory.get_remaining_memory() < program_data.len() {
        return Err("Not enough memory to load program");
    }

    self.program_queue.pop_front();
    memory.create_process(program_info, program_data);

    self.unload_list.push(program_id);

    Ok(program_id)
}

pub fn batch_step(&mut self) -> Vec<u32> {
    let mut process_ids = Vec::new();

    while !self.program_queue.is_empty() {
        match self.step() {
            Ok(process_id) => process_ids.push(process_id),
            Err(_) => break
        }
    }

    process_ids
}
```

The "step" method removes a program ID from the front of the queue and creates a process for it in memory using the memory "create_process" method. If there is not enough memory available to allocate the process, an error that can be handled is returned.

The "batch_step" method iteratively calls the "step" method until no more processes can be allocated. This effectively fills memory with as many processes as can fit.

To unload the process from memory and update the disk, the "unload_all" method is called:

```
pub fn unload_all(&mut self) {
    let mut memory = self.memory.write().unwrap();

    for program_id in self.unload_list.iter() {
        let pcb = memory.get_pcb_for(*program_id);
        let pcb = pcb.lock().unwrap();
```

```

        let data = memory.read_block_from(pcb.
            get_mem_out_start_address(), pcb.
            get_mem_end_address());
        let mut disk = self.disk.borrow_mut();

        disk.update_program(*program_id, &data);
    }

    self.unload_list.clear();
    memory.core_dump();
}

```

For each process, the output buffer and temp buffer are read from memory and then passed to the disk. This overwrites the disk so that the disk can later be written to a file.

3.5 Short-Term Scheduler

Processes in memory need to be scheduled to run on the central processing unit (CPU). The short-term scheduler (STS) is what orchestrates this.

```

pub(crate) struct ShortTermScheduler {
    resources: Arc<Mutex<ShortTermSchedulerResources>>,
    dispatch_should_terminate: Arc<AtomicBool>,
}

struct ShortTermSchedulerResources {
    cpu: Arc<Mutex<Cpu>>,
    ready_queue: Box<dyn SchedulerQueue + Send>,
    current_pcb: Option<Arc<Mutex<ProcessControlBlock>>>,
    all_procs_are_finished_condvar: Arc<(Mutex<bool>,
        Condvar)>,
}

```

The STS utilizes several resources including a reference to the CPU itself, a ready queue, which provides the next process to be executed on the CPU, and a reference to the PCB that is currently executing on the CPU. Additionally, a condition variable is kept in resources, which is used to notify threads when the ready queue is empty and nothing is running on the CPU.

A process is enqueued to the ready queue with the following method:

```

pub fn schedule_process(&mut self, pcb: Arc<Mutex<
    ProcessControlBlock>>) {
    let mut resources = self.resources.lock().unwrap();
    resources.ready_queue.push(pcb);

    let (lock, condvar) = &*resources.
        all_procs_are_finished_condvar;
    let mut all_procs_are_finished = lock.lock().unwrap();

    *all_procs_are_finished = false;
    condvar.notify_all();
}

```

The process's PCB is added to the queue and the condition variable is set to false to signify that there is at least one process in the queue. Note that the ready queue itself can be set to follow FIFO (first in, first out) or priority scheduling. Under FIFO scheduling, processes are removed from the queue in the order they were enqueued. Under priority scheduling, processes are removed based on their priority level, where a higher value means a higher priority.

A separate thread is spawned within the STS that repeatedly calls the dispatch method:

```

// Within ShortTermScheduler::new().
thread::spawn(move || {
    while !dispatch_should_terminate_clone.load(Ordering::
        Relaxed) {
        ShortTermScheduler::dispatch(&resources_clone);
    }
}

```

```

    }
});
...

fn dispatch(resources: &Arc<Mutex<
    ShortTermSchedulerResources>>) {
    let all_procs_are_finished_condvar = {
        let resources = resources.lock().unwrap();
        resources.all_procs_are_finished_condvar.clone()
    };

    {
        let (lock, condvar) = &*
            all_procs_are_finished_condvar;
        let mut all_procs_are_finished = lock.lock().unwrap()
            ();

        while *all_procs_are_finished {
            all_procs_are_finished = condvar.wait(
                all_procs_are_finished).unwrap();
        }
    }

    let (cpu, in_pcb, out_pcb) = {
        ...
    };

    let in_pcb_clone = in_pcb.clone();
    let out_pcb_clone = out_pcb.clone();
    let out_pcb_state;

    let mut cpu = cpu.lock().unwrap();

    out_pcb_state = cpu.await_process_interrupt();
    cpu.execute_process(in_pcb, out_pcb);

    if out_pcb_clone.is_none() {
        return;
    }

    let mut resources = resources.lock().unwrap();
    match out_pcb_state {
        ...
        ProcessState::Terminated => {
            let out_pcb = out_pcb_clone.unwrap();
            let mut out_pcb = out_pcb.lock().unwrap();

            out_pcb.state = ProcessState::Terminated;
            out_pcb.end_record_turnaround_time(); // Finish
                recording turnaround time.
        },
        ...
    }

    if in_pcb_clone.is_none() {
        let (lock, condvar) = &*resources.
            all_procs_are_finished_condvar;
        let mut all_procs_are_finished = lock.lock().unwrap()
            ();

        *all_procs_are_finished = true;
        condvar.notify_all();
    }
}

```

Dispatch is continuously called on its thread. Several steps occur within "dispatch". First, the thread sleeps if all processes are finished executing, meaning there is nothing currently running on the CPU and there is nothing in the ready queue.

Next, a reference to the PCB for the process that the CPU is executing is assigned to a local variable. If there is a PCB in the ready queue, it is also assigned to a local variable. The thread then blocks until the process currently on the CPU interrupts. In our implementation, this only happens when a process reaches the instruction to halt. However, this implementation allows for flexibility to further expand functionality with error interrupts, preemption, and a waiting queue.

Once the current process has been interrupted, the new

process, if it exists, is placed on the CPU while the program counter and registers on the CPU are copied into the PCB of the process that was just interrupted.

After the new process has been dispatched, the process that was moved off the CPU is handled based on its interrupt state. Finally, if there was no new process to place on the CPU, all threads are notified that all processes have finished execution. All resources used are properly protected with critical sections to ensure no race conditions.

This dispatch implementation effectively merges dispatch and context switching into one and allows for processes to be dynamically added to the ready queue at any time.

Finally, the STS provides a method to wait for all of its enqueued processes to finish:

```
pub fn await_all_procs_finished(&self) {
    let all_procs_are_finished_condvar = {
        let resources = self.resources.lock().unwrap();
        resources.all_procs_are_finished_condvar.clone()
    };

    let (lock, condvar) = &*all_procs_are_finished_condvar;
    let mut all_procs_are_finished = lock.lock().unwrap();

    while !*all_procs_are_finished {
        all_procs_are_finished = condvar.wait(
            all_procs_are_finished).unwrap();
    }
}
```

This is so that the driver can wait for the processes it enqueued to finish executing.

3.6 Central Processing Unit

The central processing unit (CPU) is responsible for executing processes. For an operating system that isn't simulated, the kernel would run on the CPU in kernel mode. However, our CPU is only responsible for executing the tasks placed on it to run.

```
pub(crate) struct Cpu {
    resources: Arc<Mutex<CpuResources>>,
    cycle_should_terminate: Arc<AtomicBool>,
    dma_should_terminate: Arc<AtomicBool>,
    dma_channel_handle: Option<thread::JoinHandle<()>>,
}

struct CpuResources {
    memory: Arc<RwLock<Memory>>,
    dma_sender: mpsc::Sender<DmaCommand>,
    cache: Vec<u32>,
    program_counter: usize,
    mem_start_address: usize,
    registers: [u32; 16],
    proc_should_interrupt_condvar: Arc<(Mutex<bool>, Condvar)>,
    proc_interrupt_type: ProcessState,
}
```

The CPU stores a resource struct and variables for thread management. Included in the resources struct are the registers, program counter, cache, a reference to memory, and the memory start address for the process (for computing physical address). There is also a condition variable used to signal interrupts and an interrupt type to signify what state a process should go into after interruption.

Our CPU operates using two concurrently running threads, the compute-only execution thread and the direct-memory-access (DMA) thread.

3.6.1 Context Switching

Moving a process onto the CPU is done through the "execute_process" method:

```
pub fn execute_process(&mut self, in_pcb: Option<Arc<Mutex<
    ProcessControlBlock>>>, out_pcb: Option<Arc<Mutex<
    ProcessControlBlock>>>) {
    ...

    let mut resources = self.resources.lock().unwrap();

    if let Some(out_pcb) = out_pcb {
        let mut out_pcb = out_pcb.lock().unwrap();

        out_pcb.program_counter = resources.program_counter;
        out_pcb.registers.copy_from_slice(&resources.registers);
        out_pcb.end_record_burst_time(); // End recording burst time.
    }

    if let Some(in_pcb) = in_pcb {
        let mut in_pcb = in_pcb.lock().unwrap();

        in_pcb.start_record_burst_time(); // Start recording burst time.

        resources.cache = {
            let memory = resources.memory.read().unwrap();
            memory.read_block_from(in_pcb.get_mem_start_address(), in_pcb.get_mem_in_start_address());
        };
        resources.program_counter = in_pcb.program_counter;
        resources.mem_start_address = in_pcb.get_mem_start_address();
        resources.registers.copy_from_slice(&in_pcb.registers);

        let (lock, condvar) = &*resources.proc_should_interrupt_condvar;
        let mut should_interrupt = lock.lock().unwrap();

        *should_interrupt = false;
        condvar.notify_all();
    }
}
```

First, the CPU's current program counter and registers are copied into the output PCB if there is one. Then the program counter and registers in the input PCB are copied onto the CPU if the input PCB exists. If there is no input PCB, then the CPU remains interrupted. Otherwise, the condition variable is used to signal the CPU to start executing the program. Additionally, the cache is populated by retrieving from memory the entire instruction buffer of the process for the input PCB.

3.6.2 CPU Cycling

The compute-only thread is the thread that sees the most utilization.

```
// Within Cpu::new().
thread::spawn(move || {
    while !cycle_should_terminate_clone.load(Ordering::
        Relaxed) {
        Cpu::cycle(&resources_clone);
    }
});

...

fn cycle(resources: &Arc<Mutex<CpuResources>>) {
    let proc_should_interrupt_condvar = {
        let resources = resources.lock().unwrap();
        resources.proc_should_interrupt_condvar.clone()
    };

    {
```

```

    let (lock, condvar) = &*
        proc_should_interrupt_condvar;
    let mut should_interrupt = lock.lock().unwrap();

    while *should_interrupt {
        should_interrupt = condvar.wait(
            should_interrupt).unwrap();
    }
}

let mut resources = resources.lock().unwrap();

let current_instruction = resources.cache[resources.
    program_counter];
resources.program_counter += 1;

let decoded_instruction = Cpu::decode(
    current_instruction);

Cpu::execute(&mut resources, &decoded_instruction);
}

```

This thread calls the “cycle” method repeatedly. The “cycle” method first sleeps if the CPU is currently in an interrupted state.

When not in an interrupted state, the CPU first fetches the current instruction. It’s prudent to mention that the CPU does not fetch instructions directly from memory. Rather, it loads the instructions for the current process from the cache. Once the instruction is retrieved, the program counter is incremented by one.

Next, the fetched instruction is decoded. Bits are extracted from the 32-bit instruction using bit-shift operations using the following method:

```

fn extract_bits(instruction: u32,
    start_index: u32,
    length: u32) -> u32 {
    (instruction << start_index) >> (32 - length)
}

```

This is how the instruction is broken down into its parts: First, the decoder extracts the first two bits from the instruction to determine the instruction type. Depending on the type (arithmetic, condition branch immediate, unconditional jump, or IO), the decoder extracts numbers of bits representing any register numbers or memory addresses/data elements the instruction contains. These results are stored in a struct:

```

struct DecodedInstruction {
    instr_type: u8,
    opcode: u8,
    reg_1_num: usize,
    reg_2_num: usize,
    reg_3_num: usize,
    address: usize,
}

```

Once the instruction is decoded, the “execute” method is called. Execution is split into four methods, each corresponding to one of the four instruction types. Handled separately is the no-op instruction, which has no instruction type. Each method contains a match statement (Rust’s equivalent of switch) which takes the opcode and performs the specific instruction execution task required by that opcode, as defined by the project specifications.

3.6.3 Direct Memory Access

The direct memory access (DMA) channel is a middleman between the compute-only part of the CPU and the main memory. When an IO operation is executed, the memory access is delegated to the DMA thread:

```

running 4 tests
test kernel::cpu::tests::test_execute_job1 ... ok
test kernel::cpu::tests::test_execute_job3 ... ok
test kernel::cpu::tests::test_execute_job2 ... ok
test kernel::cpu::tests::test_execute_job4 ... ok

successes:

successes:
  kernel::cpu::tests::test_execute_job1
  kernel::cpu::tests::test_execute_job2
  kernel::cpu::tests::test_execute_job3
  kernel::cpu::tests::test_execute_job4

test result: ok. 4 passed; 0 failed; 0 ignored; 0 measured; 28 filtered
out; finished in 0.10s

```

Fig. 2. Output of the CPU tests running

```

// Within Cpu::new().
let (dma_sender, dma_receiver) = mpsc::channel();

let dma_channel_handle = thread::spawn(move || {
    while !dma_should_terminate_clone.load(Ordering::
        Relaxed) {
        if let Ok(command) = dma_receiver.recv_timeout(
            Duration::from_millis(100)) {
            match command {
                DmaCommand::Fetch { address,
                    response_sender } => {
                    let memory_clone = memory_clone.read().
                        unwrap();
                    let value = memory_clone.read_from(
                        address);
                    response_sender.send(value).unwrap();
                },
                DmaCommand::Store { address, value,
                    response_sender } => {
                    let mut memory_clone = memory_clone.
                        write().unwrap();
                    memory_clone.write_to(address, value);
                    response_sender.send().unwrap();
                },
            }
        }
    }
});

```

This thread waits to be signaled. When a command is received, the thread performs the desired memory operation and sends a message back to the calling thread. It is important to note that our operating system simulation has no long-term IO devices. Because of this, the DMA channel functions merely as a proof of concept and does not have any real utility. It is for this same reason that there is no waiting queue in the short-term scheduler. In a real system, an IO operation would be delegated to the DMA channel while the process which called DMA would be placed on the waiting queue. Other processes would execute on the CPU while the DMA channel completes its operation.

4 SIMULATION

The programs to be executed to simulate operating system usage were 30 handmade jobs. The structure of the programs can be seen in Figure 1. To ensure that the system could execute the jobs correctly, the first four jobs were thoroughly tested in CPU unit tests. The output for those unit tests can be seen in Figure 2

To test system completeness, the driver was set up to a) load all 30 jobs into the disk from the program file, b) enqueue all 30 jobs into the long-term scheduler, c) batch step

```

Starting the driver.
Loading programs into disk.
Enqueuing all programs into LTS.
Starting the LTS:
...Batch 1:
.....Loading programs into memory.
.....Scheduling 15 processes into STS.
.....Awaiting all scheduled processes to finish.
.....Writing output buffer and temp buffer for 15 processes to disk.
.....Unloading all processes from memory.
...Batch 2:
.....Loading programs into memory.
.....Scheduling 15 processes into STS.
.....Awaiting all scheduled processes to finish.
.....Writing output buffer and temp buffer for 15 processes to disk.
.....Unloading all processes from memory.

```

Fig. 3. Example output of the simulation running

```

Stats for executed processes (FIFO Scheduling):
... ID | Priority | Turnaround Time (ms) | Avg Burst Time (ms)
... ---|-----|-----|-----
... 01 | 02      | 00.20                | 00.12
... 02 | 04      | 00.26                | 00.07
... 03 | 06      | 00.33                | 00.07
... 04 | 05      | 00.38                | 00.05
... 05 | 03      | 00.43                | 00.06
... 06 | 07      | 00.49                | 00.05
... 07 | 05      | 00.54                | 00.05
... 08 | 16      | 00.58                | 00.05
... 09 | 02      | 00.63                | 00.05
... 10 | 10      | 00.69                | 00.05

```

Fig. 4. Process runtime metrics using FIFO scheduling

the long-term scheduler to load 15 programs into memory as processes, d) enqueue the 15 processes of the current batch into the short-term scheduler and subsequently execute them, and e) print out the timing data once the processes finished and write the updated disk to a file. Figure 3 shows the console output of the simulation running.

Loading 15 programs into memory as processes was enforced by the design requirement that the memory be half the size of the disk. This guarantees that only half of the programs can be loaded into memory at a time due to the disk being the exact size to hold the entirety of the program file.

The simulation also outputs the timing data for the processes including the ID, priority level, turnaround time, and burst time of each process. The turnaround time represents the time from which the process was created in memory

```

Stats for executed processes (Priority Scheduling):
... ID | Priority | Turnaround Time (ms) | Avg Burst Time (ms)
... ---|-----|-----|-----
... 01 | 02      | 00.99                | 00.05
... 02 | 04      | 00.80                | 00.07
... 03 | 06      | 00.55                | 00.07
... 04 | 05      | 00.72                | 00.05
... 05 | 03      | 00.92                | 00.07
... 06 | 07      | 00.17                | 00.11
... 07 | 05      | 00.59                | 00.05
... 08 | 16      | 00.22                | 00.05
... 09 | 02      | 01.03                | 00.05
... 10 | 10      | 00.40                | 00.07

```

Fig. 5. Process runtime metrics using Priority scheduling

TABLE 1
Process Runtimes (FIFO Scheduling)

ID	Priority	Turnaround Time (ms)	Burst Time (ms)
1	2	0.19	0.08
2	4	0.24	0.06
3	6	0.31	0.06
4	5	0.35	0.05
5	3	0.43	0.08
6	7	0.53	0.10
7	5	0.59	0.06
8	16	0.65	0.06
9	2	0.66	0.06
10	10	0.72	0.06
11	3	0.78	0.06
12	9	0.85	0.07
13	12	0.90	0.06
14	2	0.94	0.04
15	5	0.99	0.05
16	8	0.07	0.03
17	4	0.13	0.06
18	12	0.20	0.06
19	1	0.25	0.05
20	2	0.31	0.07
21	4	0.36	0.05
22	10	0.43	0.07
23	5	0.48	0.05
24	7	0.53	0.06
25	9	0.57	0.04
26	4	0.60	0.03
27	9	0.66	0.06
28	3	0.72	0.06
29	1	0.77	0.05
30	8	0.83	0.06

Turnaround Time

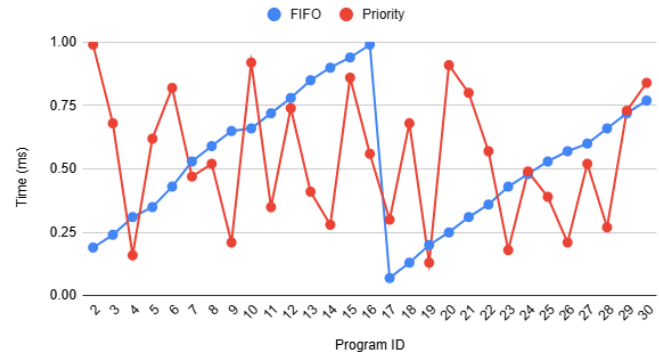


Fig. 6. Process ID vs Runtime for priority and FIFO scheduling

to the time the process finished its execution. The average burst time was the average time the process spent on the CPU each time it was on the CPU. Since each process was on the CPU only a single time, the burst time indicates how long it took for the CPU to complete every instruction. See Figure 4 for a FIFO scheduling output and Figure 5 for a priority scheduling output.

5 ANALYSIS

A couple of deductions can be made from Table 1 and Table 2. The first thing to notice is the burst time is low and similar for every process. This is due to every program in the program file having a similar number of instructions.

TABLE 2
Process Runtimes (Priority Scheduling)

ID	Priority	Turnaround Time (ms)	Burst Time (ms)
1	2	0.99	0.06
2	4	0.68	0.06
3	6	0.16	0.09
4	5	0.62	0.05
5	3	0.82	0.08
6	7	0.47	0.06
7	5	0.52	0.05
8	16	0.21	0.06
9	2	0.92	0.06
10	10	0.35	0.07
11	3	0.74	0.06
12	9	0.41	0.06
13	12	0.28	0.07
14	2	0.86	0.04
15	5	0.56	0.05
16	8	0.30	0.02
17	4	0.68	0.10
18	12	0.13	0.08
19	1	0.91	0.05
20	2	0.80	0.06
21	4	0.57	0.05
22	10	0.18	0.06
23	5	0.49	0.09
24	7	0.39	0.05
25	9	0.21	0.04
26	4	0.52	0.04
27	9	0.27	0.06
28	3	0.73	0.06
29	1	0.84	0.05
30	8	0.33	0.04

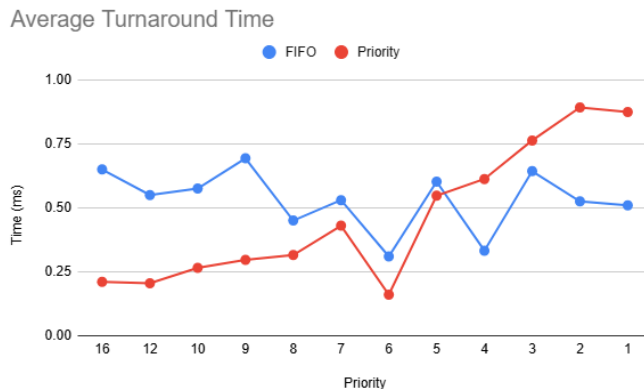


Fig. 7. Priority vs Runtime for priority and FIFO scheduling

The second deduction that can be made from the team's data can be seen when comparing turnaround times between FIFO and priority scheduling. Figure 6 shows turnaround increasing linearly as the process ID increases for FIFO scheduling, but an erratic turnaround time for priority scheduling. Figure 7 is plotted using the same data but is instead organized by the priority. Under priority scheduling, the higher-priority jobs have a faster turnaround time. Meanwhile, under FIFO the priority does not correlate with turnaround time.

6 CONCLUSION

Due to this project's original scope being larger than the final scope, the team must come to a different conclusion than

originally intended. As such, there is no multi-core variant of the operating system simulation. Without two variants of the simulator, the single version must be compared against itself. This comparison can be used to determine how well the simulator acts as a standalone example of how a modern operating system functions.

From the data, the implementation for handling two types of scheduling algorithms was successful, showing two linear trends for turnaround time. The turnaround and burst times themselves, alongside the accurate completion of all 30 jobs from the program file, show that Rust as a programming language and the system design itself combine to form a strong method for creating and designing an operating system simulator.

When considering this report's contents, the package is an acceptable and functional operating system simulator. If allocated the time, the current state of the project would act as a sufficient baseline for creating a variant that operates with multiple cores, a program file requiring a wait state, and context switching. Without these aspects, the simulator is limited to being entirely single-core reliant.

REFERENCES

- [1] A. Silberschatz, P. B. Galvin, and G. Gagne, *Operating System Concepts*. Wiley, 2018.